Assignment 10: Leader–Follower vs Leaderless KV Latency

Yalin Sun

1 Setup

We benchmark a five-node key–value service implemented in Go. Three *leader–follower* (LF) modes differ by read/write quorum sizes (R, W): **LF W=5,R=1** (reads from the leader only; writes replicate to all followers), **LF W=1,R=5** (writes touch one replica; reads quorum all five), and **LF W=3,R=3** (balanced quorum). A **leaderless** mode uses $W{=}N$ and $R{=}1$ with Lamport-style versions (similar to Dynamo-style sloppy quorum with a single-replica read coordinator in our client path). Artificial sleeps model network/processing cost: followers sleep on replicate and internal read, and the leader sleeps after notifying followers.

The load generator issues mixed read/write traffic at four target write fractions: 1%, 10%, 50%, and 90%. For each run we record read latency, write latency, and the elapsed time between a read and the most recent write to the *same* key ("read–write gap"). Figures show empirical CDFs; all four configurations are overlaid on one plot per metric and workload mix.

2 How to reproduce the figures

From the `kv-service` directory:

```
python scripts/plot_report_comparisons.py
```

This reads `results/<config>/wr*.jsonl` and writes the PNGs under `results/figures/report/`. To regenerate single-configuration histograms/CDFs, use `scripts/plot_results.py` with each JSONL file and `-o results/figures/<config>/`.

3 Results by read/write ratio

3.1 1% writes (read-heavy)

Reads. Leaderless and **LF W=5,R=1** sit on the left of the CDF: both effectively do a single-hop read (leaderless $R{=}1$; LF reads only the leader). **LF W=1,R=5** and **LF W=3,R=3** are slower because every read must wait for multiple replicas (fan-out plus follower "internal read" delays), so their CDFs shift right by tens of milliseconds.

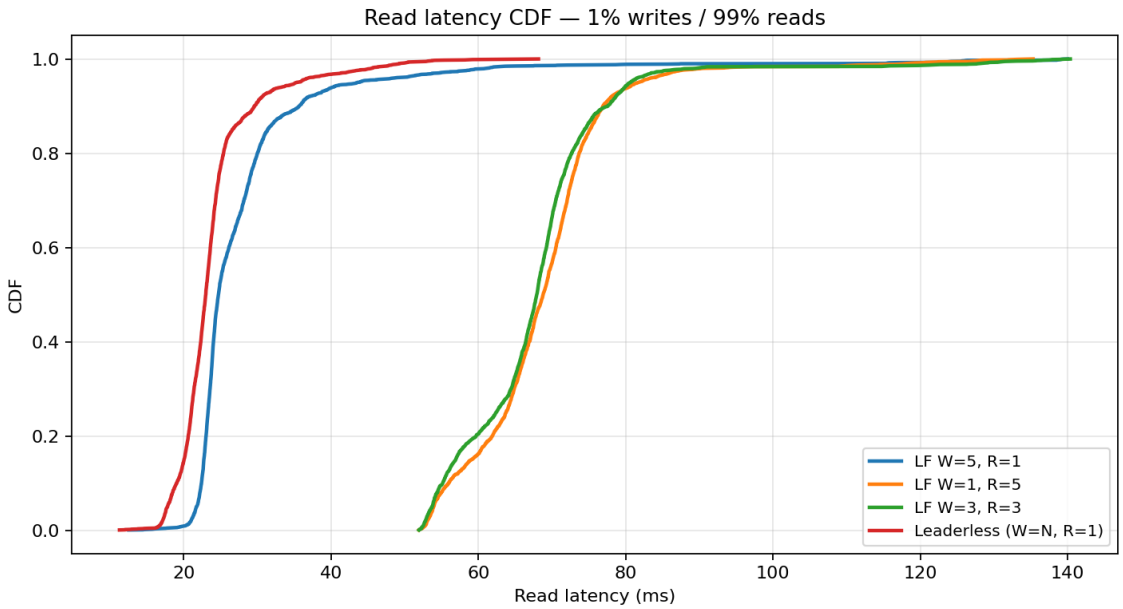

Figure 1: Read latency at 1% writes.

Writes. LF W=1,R=5 is fastest: the write path updates one replica first. **LF W=3,R=3** is in the middle (partial quorum). **LF W=5,R=1** and **leaderless** are both slow here because each write must replicate to all nodes before returning—the leader waits for every follower in LF; leaderless coordinates a full $W{=}N$ write.

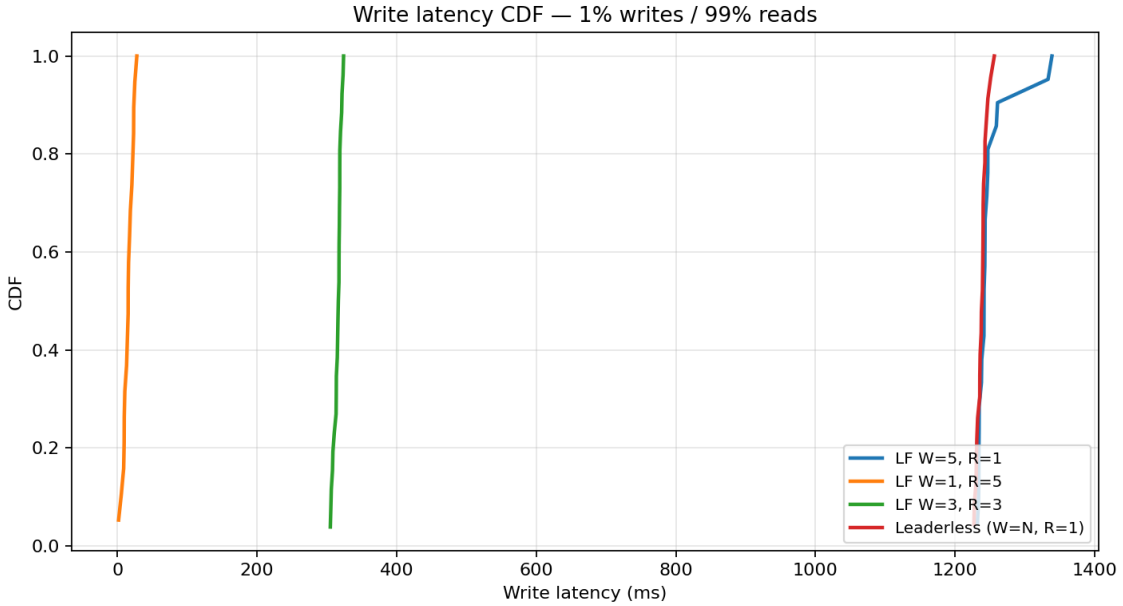

Figure 2: Write latency at 1% writes.

Read–write gap. With almost all traffic reads, many reads hit keys whose last write was long ago; the gap distribution is dominated by inter-arrival spacing rather than replication semantics.

Differences between modes are modest compared to write-heavy mixes.

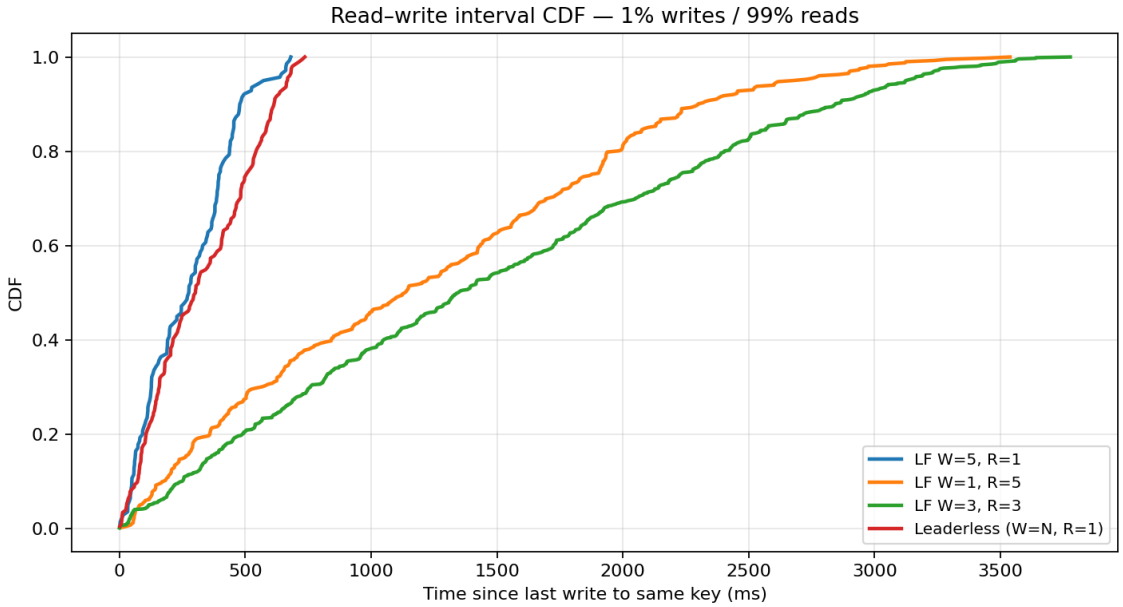

Figure 3: Read–write interval at 1% writes.

Which LF is "best" here? If the product cares about *read* tail latency under extreme read skew, **leaderless** or **LF W=5,R=1** wins. If occasional writes must be cheap (e.g., rare configuration updates), **LF W=1,R=5** is better on write latency at the cost of expensive reads—a poor fit for this 99% read workload.

3.2 10% writes

Reads. Leaderless and **LF W=5,R=1** still form the fast cluster on the left of the CDF; in our numbers leaderless keeps a slightly lower median than LF W=5,R=1 at 10% writes, while both remain far faster than the $R>1$ modes. The gap between leaderless and the leader-read path is small because both are essentially single-replica reads in this implementation.

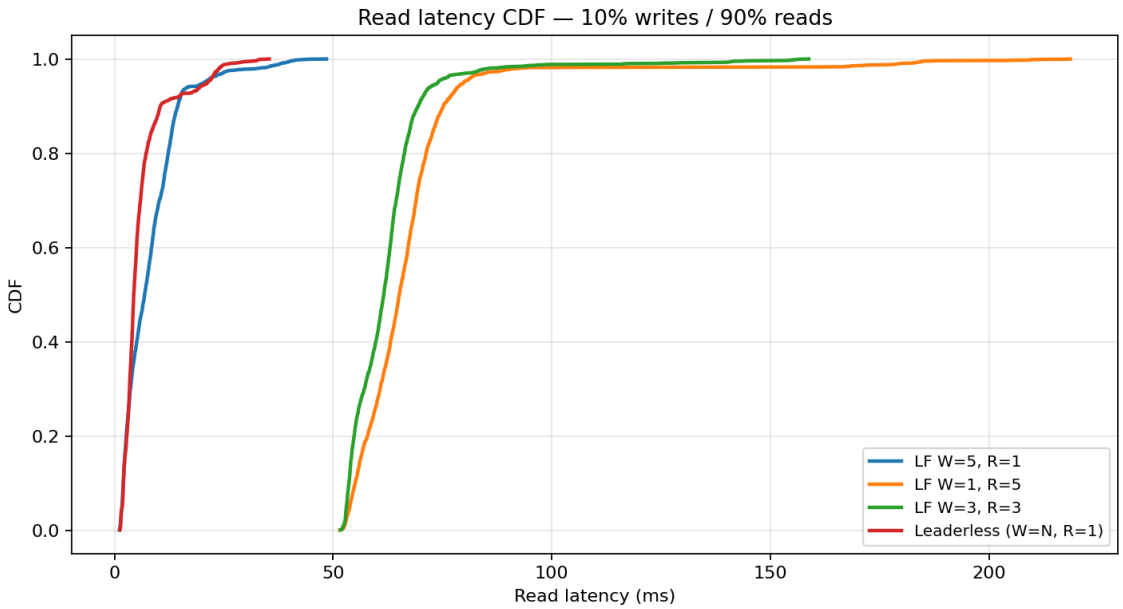

Figure 4: Read latency at 10% writes.

Writes. Ordering is unchanged in spirit: **LF W=1,R=5** remains the write winner; full-replication modes (**LF W=5,R=1**, **leaderless**) stay near ∼1.2 s medians because of the modeled all-replica delay.

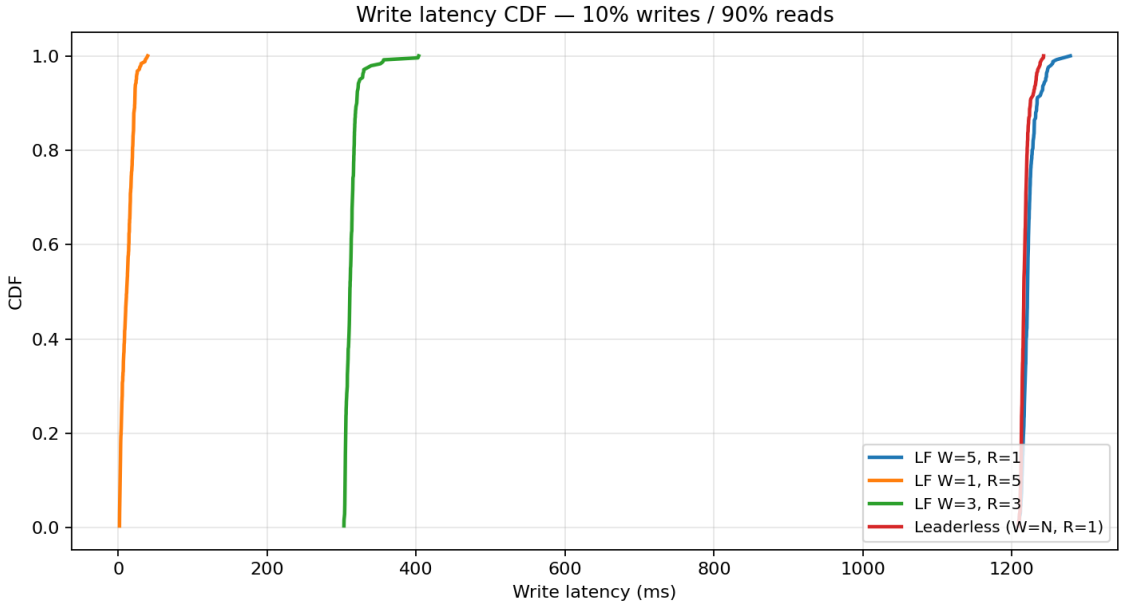

Figure 5: Write latency at 10% writes.

Gap. More writes shorten typical gaps—reads more often follow a recent write on the same key—so the gap CDF moves left relative to the 1% case.

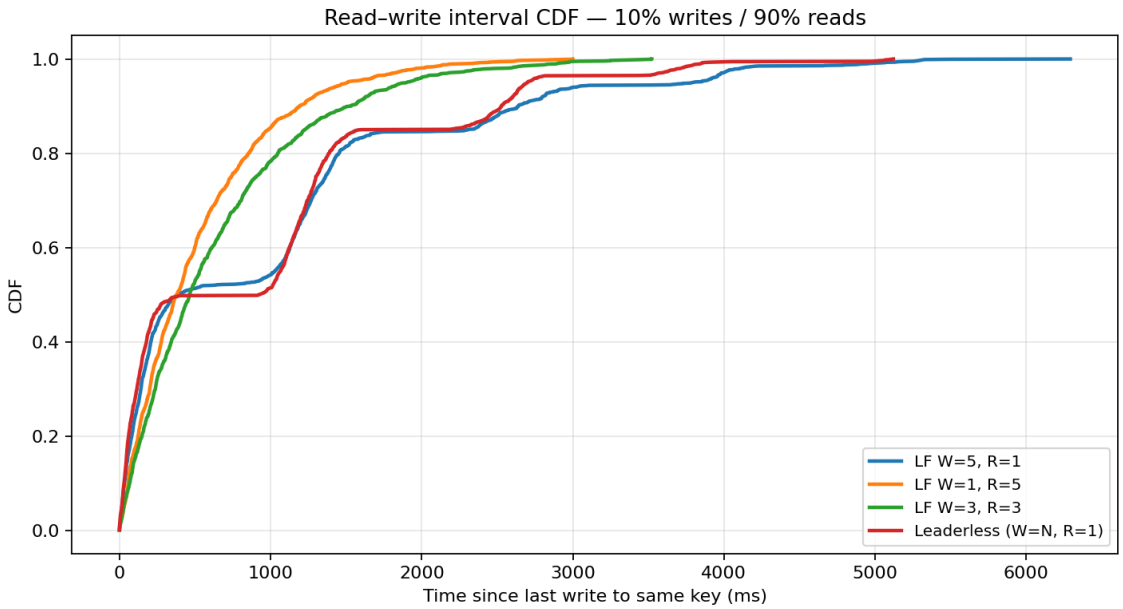

Figure 6: Read–write interval at 10% writes.

Best trade-off. For 10% writes, **LF W=5,R=1** is attractive if you want low read latency while accepting slow, strongly replicated writes. **LF W=1,R=5** only makes sense if the business truly prioritizes write latency over read latency (unusual at this read skew).

3.3 50% writes

With balanced traffic, the tension between "cheap read" and "cheap write" is obvious. **LF W=5,R=1** keeps reads fastest; **LF W=1,R=5** keeps writes fastest; **LF W=3,R=3** compromises both. **Leaderless** behaves like a full-write system with R=1 reads—writes stay expensive, reads stay relatively good but not as dominant as under pure read skew.

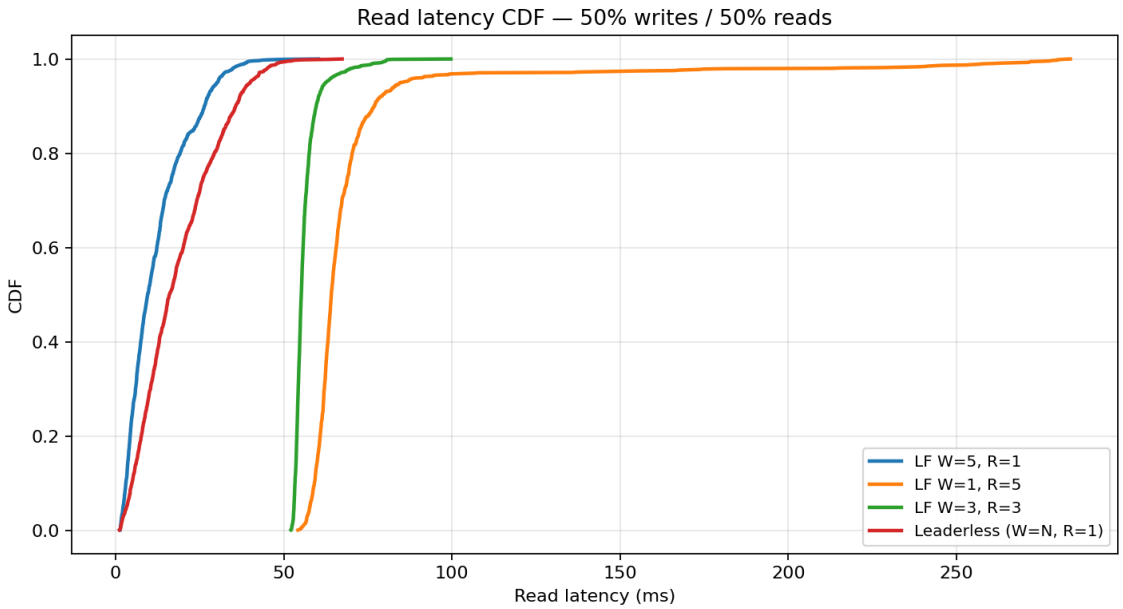

Figure 7: Read latency at 50% writes.

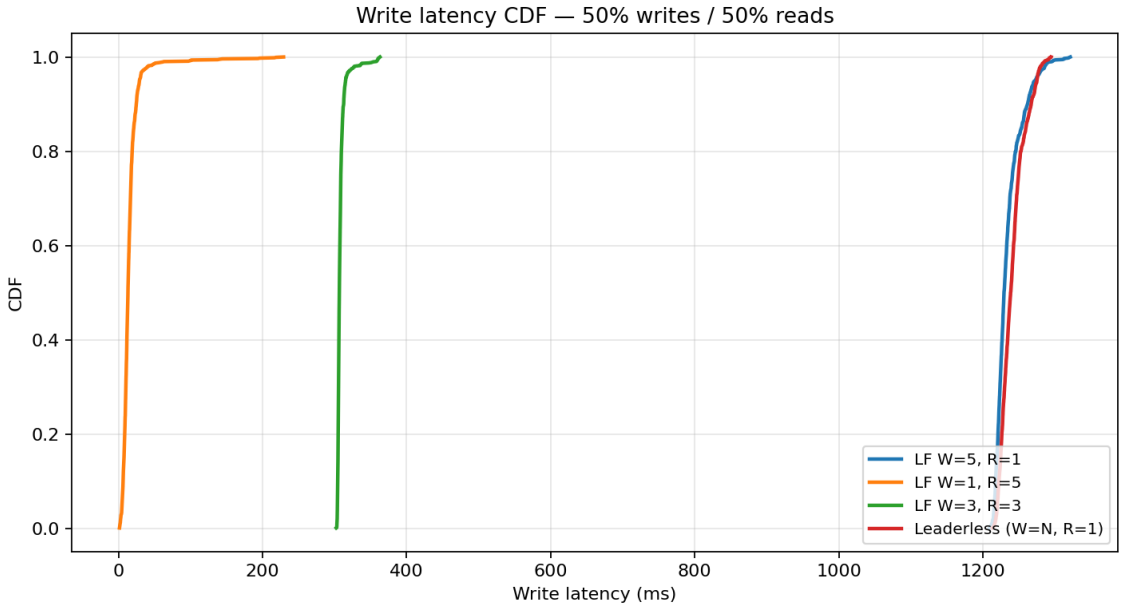

Figure 8: Write latency at 50% writes.

The read–write gap distribution tightens further: half the operations are writes, so a read often follows a write on the same key within a short window.

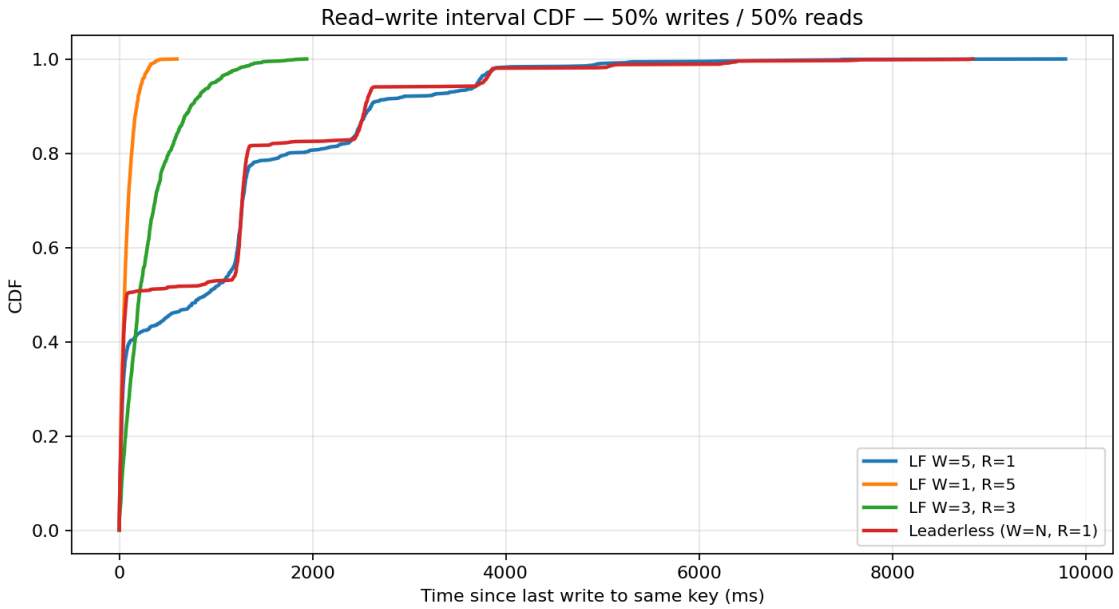

Figure 9: Read–write interval at 50% writes.

With balanced read/write traffic, no single configuration dominates both dimensions. **LF W=5,R=1** retains the lowest read latency, while **LF W=1,R=5** keeps writes cheapest—but each wins on only one axis. **LF W=3,R=3** is the most sensible overall choice: it is not the fastest at either reads or writes, but it avoids the worst-case penalty on both, making it the natural balanced option when neither reads nor writes can be deprioritized.

3.4 90% writes (write-heavy)

Reads. Under heavy write contention, **LF W=5,R=1** still wins on reads: the leader serves reads while writes pipeline through the same node, but a single-replica read avoids the multi-round quorum cost of $R>1$ LF modes.

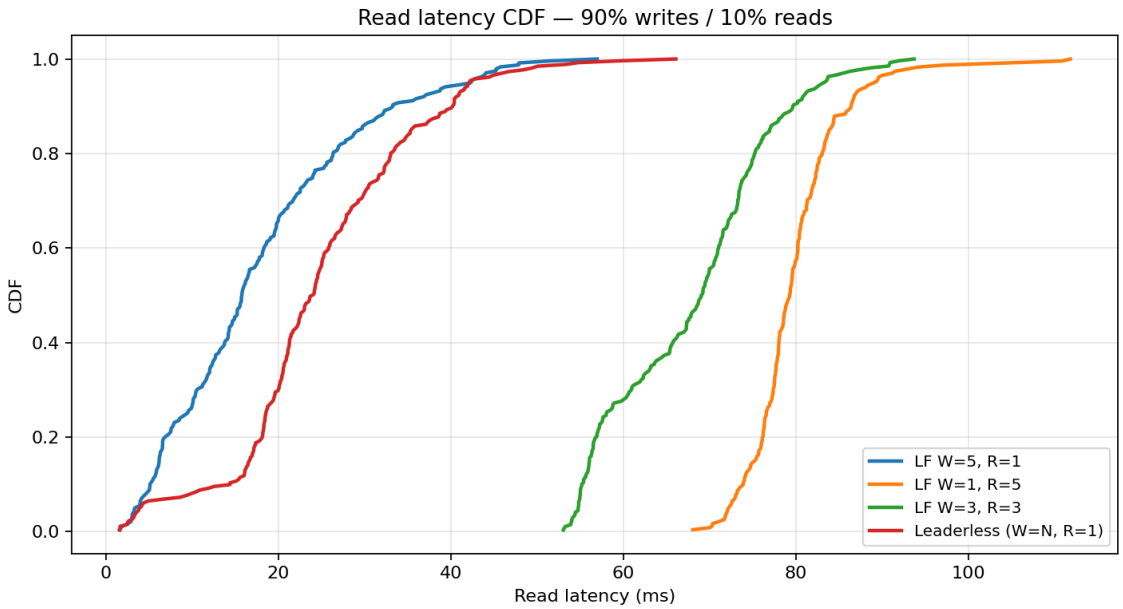

Figure 10: Read latency at 90% writes.

Writes. LF W=1,R=5 is clearly best—only one replica must complete before the client sees success (in our model). Full replication (**LF W=5,R=1**, **leaderless**) pays the full follower chain on almost every operation.

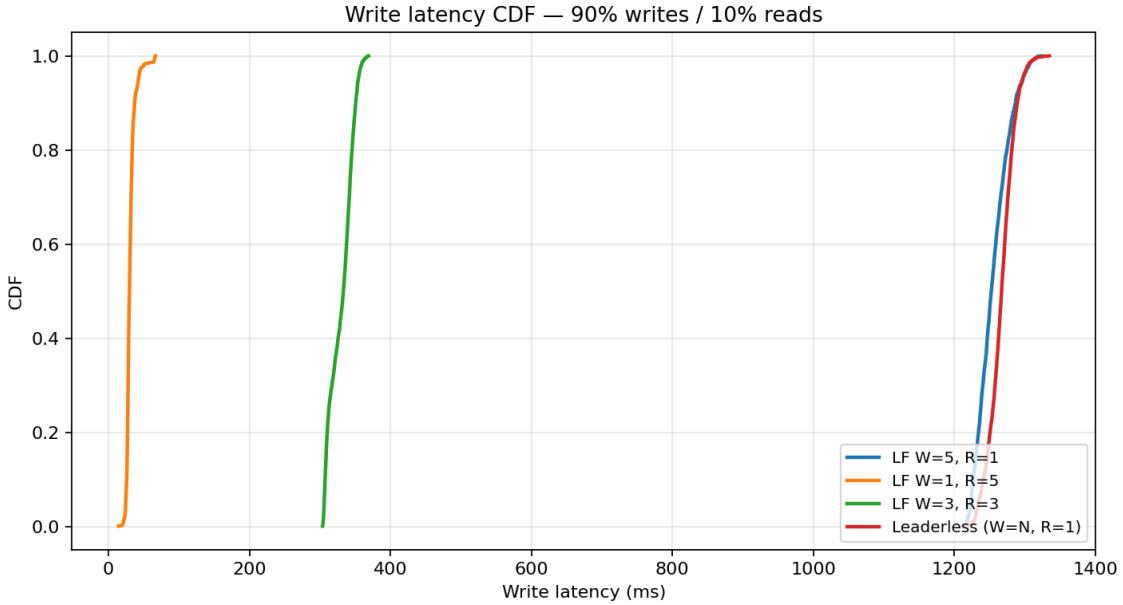

Figure 11: Write latency at 90% writes.

Gap. Gaps become very short: most reads immediately follow a write to the same key, so the gap CDF is concentrated at small latencies; differences reflect queueing and which replica was last written.

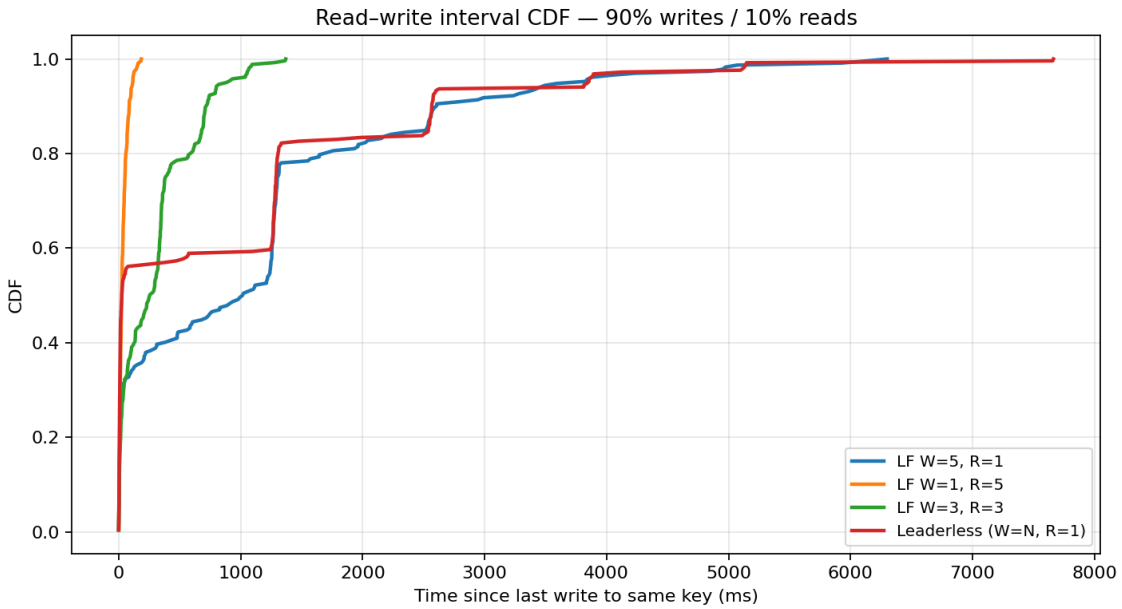

Figure 12: Read–write interval at 90% writes.

Winner for write-heavy OLTP-style workloads: LF W=1,R=5 on write latency; if you also need low read latency in the same deployment, you would need a different architecture (read replicas, caching, or lowering R selectively) because high R hurts reads.

4 Summary: which configuration for which application?

Application profile	Sensible choice
Read-mostly content / CDN metadata	Leaderless or LF W=5,R=1; avoid high R
Mixed read/write product catalog	LF W=5,R=1 or LF W=3,R=3 depending on SLA
Write-heavy ingestion, event log	LF W=1,R=5 or partial W with async replication
Need strong durability on every write	LF W=5,R=1 or leaderless $W{=}N$ (expect slow writes)
Need quorum reads for freshness	LF W=3,R=3 or LF W=1,R=5 (pay read latency)

Takeaway. There is no single "best" leader–follower policy: low read latency wants small R and a stable read source (leader or single replica); low write latency wants small W. Our experiment makes that cost visible: modeled full replication pins write medians near 1.2–1.3 s, while $W{=}1$ writes finish in tens of milliseconds. Quorum settings split the difference but never beat specialized endpoints on both dimensions at once.